ЗВІТ

про виконання лабораторної роботи №3

«Розмітка інтерфейсу (Layout Widgets)»

з дисципліни

«Програмування мобільних додатків»

Студента групи КН-2327Б

Багрія-Белза Андрія

Завдання: Створити екран Flutter-застосунку згідно Варіанту 1 (Група 1: «Менеджер завдань», макет «Картка зображення зверху: Column → Row») без використання списків та навігації, застосувавши обов'язкові layout-віджети: Column, Row, Container, Padding, Expanded, Flexible та Spacer.

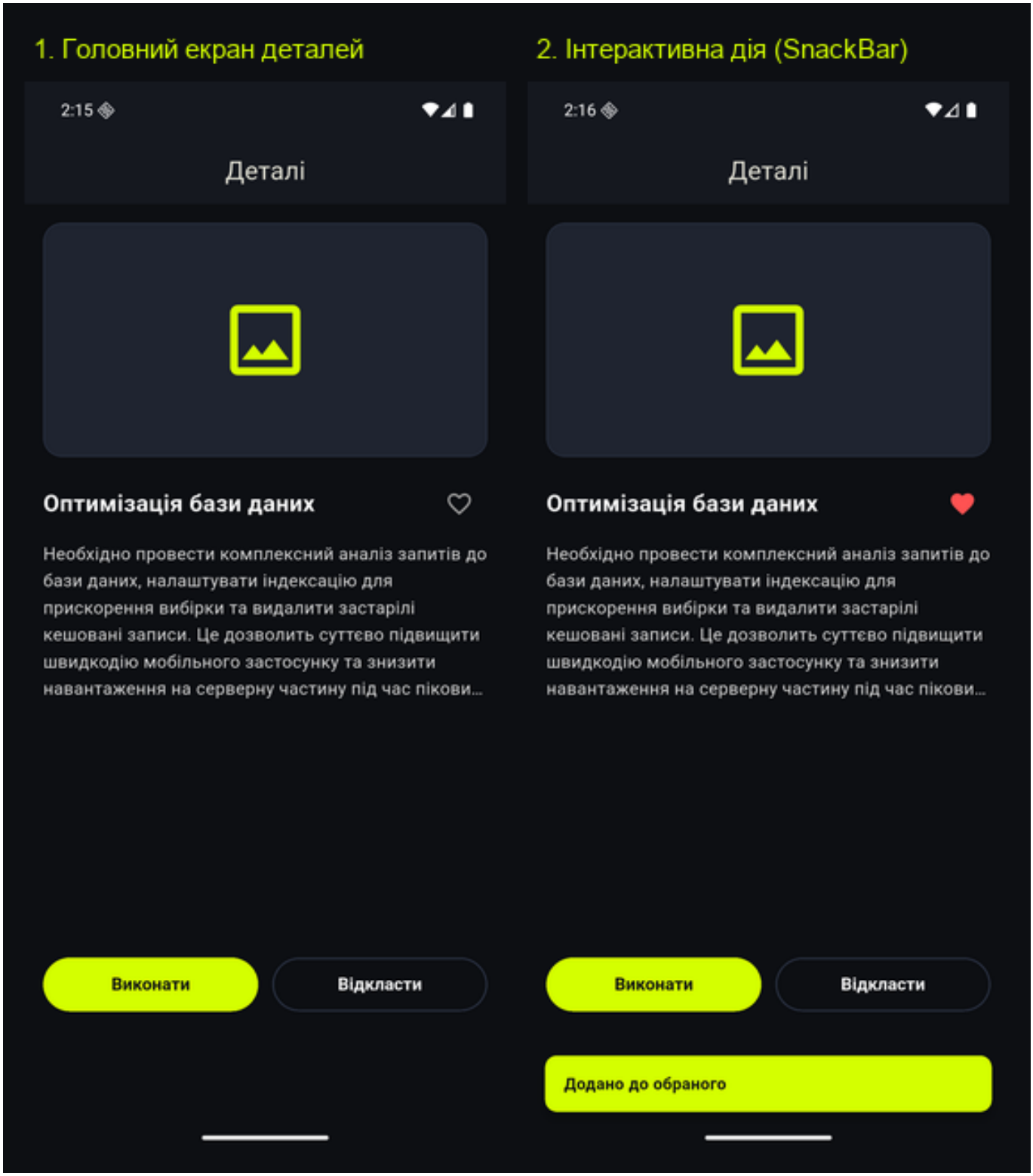
1 Написання коду-розв’язку завдання:

```
class _TaskDetailsScreenState extends State<TaskDetailsScreen> {
  bool _isFavorite = false;

  void _showMessage(String text) {
    final messenger = ScaffoldMessenger.of(context);
    messenger.hideCurrentSnackBar();
    messenger.showSnackBar(SnackBar(content: Text(text)));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Деталі'), centerTitle: true),
  // ... (пропуск неістотних рядків)
                  IconButton(
                    icon: Icon(
                      _isFavorite ? Icons.favorite : Icons.favorite_border,
                      color: _isFavorite ? Colors.redAccent : Colors.grey,
                    ),
                    onPressed: () {
                      setState(() {
                        _isFavorite = !_isFavorite;
                      });
                      _showMessage(
                        _isFavorite ? 'Додано до обраного' : 'Видалено з обраного',
                      );
                    },
                  ),
  // ... (пропуск неістотних рядків)
}
```

2 Результат виконання коду:



1. Що таке розмітка інтерфейсу у Flutter? Розмітка інтерфейсу (Layout) у Flutter — це процес просторового розташування, вирівнювання та визначення розмірів елементів за допомогою ієрархії вкладених layout-віджетів за правилом передачі обмежень зверху вниз.

2. У чому різниця між віджетами Row та Column? Віджет Row розміщує свої дочірні елементи горизонтально в один рядок, тоді як Column компонує їх вертикально у стовпчик уздовж своєї головної осі.

3. Яке призначення віджета Container? Container — універсальний віджет, що поєднує можливості задання розмірів (width, height), внутрішніх і зовнішніх відступів, а також стилізації фону, меж і радіуса заокруглень.
4. Для чого використовується віджет Padding? Padding використовується для створення внутрішніх відступів навколо свого дочірнього елемента без додавання фону чи додаткового візуального оформлення.
5. Як працює віджет Expanded? Expanded змушує дочірній елемент зайняти весь вільний доступний простір уздовж головної осі в Row або Column пропорційно до встановленого значення коефіцієнта flex.
6. Чим відрізняється Expanded від Flexible? Віджет Expanded вимагає обов'язкового заповнення всього виділеного простору (FlexFit.tight), а Flexible дозволяє займати лише потрібний розмір у межах виділеного простору (FlexFit.loose).
7. Чому не бажано використовувати фіксовані розміри елементів? Фіксовані розміри призводять до помилок переповнення екрана (RenderFlex overflowed) на невеликих пристроях або до порожніх неадаптивних зон на великих екранах.
8. Як layout-віджети допомагають створювати адаптивний інтерфейс? Вони динамічно перерозподіляють доступний екранний простір на основі відносних пропорцій і поточних розмірів вікна без жорсткої прив'язки до фізичних пікселів.
Висновок: На цій лабораторній роботі, я опанував принципи побудови адаптивних макетів користувацького інтерфейсу у Flutter, створив екран детального перегляду завдання за Варіантом 1 (Група 1) із використанням віджетів Column, Row, Container, Padding, Expanded, Flexible та Spacer, а також перевірив коректність відображення та інтерактивних дій на емуляторі Android.